

Third Biweekly Report (April 27 – May 17, 2026)

Reporting Period: April 27 – May 17, 2026 (Weeks 5–7) | Submission: May 17, 2026

1. Team Summary — WBS Progress Overview

This report covers Weeks 5–7 (April 27 – May 17), the most execution-intensive phase of the HeartCompass project. The team completed the entire WP2 Persona Building pipeline (WP2.1–WP2.5: 5/5 tasks at 100%), advanced WP3 Conversation System through core implementation (4 tasks, 75% avg. completion), and initiated WP4 Channel Integration (3 tasks, 75% avg.) and WP5 CLI/Deployment (1 task, 75%). Every task maps to a defined RBS responsibility area. Total: 161 hours across 13 WBS tasks, weighted completion 84.5%. All milestones on track.

1.1 WBS Task Progress Matrix (Weeks 5–7)

WBS Task	Description	Owner	Hours	Compl.	Status
WP2.1	FRBuildingGraph Preprocessing Pipeline	Leishiyu Chen	8	100%	Complete
WP2.2	FineGrainedFeed Extraction (4 dimensions)	Leishiyu Chen	20	100%	Complete
WP2.3	Conflict Detection & Update Logging	Leishiyu Chen	12	100%	Complete
WP2.4	Vector Embedding & Semantic Recall	Haojie Hu	20	100%	Complete
WP2.5	Persona Building Review & Test	Haojie Hu	12	100%	Complete
WP3.1	ConversationGraph State & Parallel Recall	Yihan Liu	22	100%	Complete
WP3.2	Memory Trimming & Rolling Summary	Yihan Liu	16	90%	In Progress
WP3.3	LLM Call & Dynamic Prompt Assembly	Yihan Liu	8	70%	In Progress
WP3.4	Feed Sync to FR Core Fields	Yihan Liu	4	40%	In Progress
WP4.1	Lark WebSocket Connection Layer	Leishiyu Chen	5	80%	In Progress
WP4.2	Bot Menu & Persona Commands	Tian Bu	14	85%	In Progress
WP4.3	Message Batching & Card Templates	Tian Bu	8	60%	In Progress
WP5.1	CLI Framework: Doctor & Setup Commands	Tian Bu	12	75%	In Progress

1.2 Overall Assessment & Outlook

Overall: WP2 passed all quality gates (WP2.5 review confirmed extraction accuracy across 4 dimensions and 5 FigureRole types). WP3 core pipeline (WP3.1–WP3.2) is operational with checkpoint persistence. WP3.3–WP3.4 are tracking toward Week 8 completion. WP4.1 WebSocket connectivity is functional; WP4.2 bot commands are interactive; WP4.3 batching is in progress. WP5.1 CLI is scaffolded (doctor, setup, log commands with --json support). Risk level is LOW. Next period (Weeks 8–9, May 18–31): complete WP3.3–WP3.5, WP4.3–WP4.5, WP5.2; commence WP5.4 system integration testing.

1.3 Effort Distribution by WBS Area

WP2 (Persona Building): 72h (44.7%) | WP3 (Conversation): 50h (31.1%) | WP4 (Channel): 27h (16.8%) | WP5 + Governance: 12h (7.4%)

WP2 Persona Building		72h	72h
WP3 Conversation System		50h	50h
WP4 Channel Integration		27h	27h
WP5 CLI + Governance		12h	12h

1.4 Weekly Milestone Achievement

Week	Key Deliverables (WBS)	Description	Status
Week 5 (Apr 27–May 3)	WP2.1–WP2.3	FineGrainedFeed 4-dimension extraction, conflict detection, prompt module extraction.	Achieved
Week 6 (May 4–10)	WP2.4–WP2.5 + WP3.1–WP3.2	pgvector recall operational, persona review passed, ConversationGraph with parallel recall + memory trimming.	Achieved
Week 7 (May 11–17)	WP3.3–WP3.4 + WP4.1–WP4.3 + WP5.1	LLM integration, feed sync initiation, Lark WebSocket + bot menu + batching + CLI framework.	On Track

1.5 Key Risks & Issues Log

- **PostgresSaver serialization (resolved, May 9):** ORM entity JSON serialization failure blocked WP3.2 checkpoint persistence. Fixed via custom encoder with entity-to-dict conversion. No schedule impact.
- **Card template field mapping (resolved):** Lark card template fields misaligned with FRBuildingGraphReport schema, causing blank sections. Reverted, corrected field contracts, re-implemented. ~2 days rework.
- **Memory trim boundary condition (mitigated):** Edge case where trimmer handled single-message rounds at threshold boundary. Added round-completeness check. Long-session validation in Week 8.
- **Concurrent FRBuildingGraph execution (mitigated):** Race condition risk on simultaneous /build_persona calls. Mitigated with asyncio.Semaphore(1) per-FR guard. Production monitoring planned.

Leishiyu Chen

RBS Area: Data Layer & Channel Integration (RBS R1.2.1, R2.2.1) | Total Hours: 45h

I. WBS Task Breakdown

WBS Task	Description	Hours	Compl.	Status
WP2.1	FRBuildingGraph Preprocessing Pipeline	8	100%	Complete
WP2.2	FineGrainedFeed Extraction (4 dimensions)	20	100%	Complete
WP2.3	Conflict Detection & Update Logging	12	100%	Complete
WP4.1	Lark WebSocket Connection & Message Routing	5	80%	In Progress

II. Period Summary

My primary focus was completing the WP2 data foundation (WP2.1–WP2.3, 40h, all at 100%) and initiating Lark WebSocket integration (WP4.1, 5h, 80%). The FRBuildingGraph pipeline is production-ready: typed OriginalSource inputs, FineGrainedFeed extraction across all 4 dimensions (PERSONALITY, INTERACTION_STYLE, PROCEDURAL_INFO, MEMORY), semantic contradiction detection, and auditable mutation logging via FROverallUpdateLog. Concurrency is protected by Semaphore(1). The Lark WebSocket foundation with open_id-based user-to-FR mapping and event routing is established. All WP2 deliverables passed the WP2.5 quality review.

III. Key Progress by WBS Task

- ▶ [WP2.1 · 8h · 100%] **Preprocessing & OriginalSource Persistence** Completed FRBuildingGraph preprocessing with OriginalSource persistence and source type classification (NARRATIVE_FROM_USER, SELF_LONG_FORM, CLOSE_RELATION_PRIVATE_CHAT) based on FigureRole context. Added content sanitization for malformed/empty input. Integrated with UserService for authentication-aware input tagging. Delivered on schedule.
- ▶ [WP2.2 · 20h · 100%] **FineGrainedFeed Extraction Across 4 Dimensions** Built extraction logic spanning PERSONALITY (traits/values/preferences), INTERACTION_STYLE (communication patterns, conflict approaches), PROCEDURAL_INFO (work methods, decision processes, habits), and MEMORY (life events, shared experiences). Each dimension uses dedicated LLM prompts with structured output schemas. Implemented dimension-aware upsert with key-level deduplication and graceful degradation for incomplete LLM outputs.
- ▶ [WP2.3 · 12h · 100%] **Conflict Detection & Audit Logging** Designed FineGrainedFeedConflict tracking with semantic contradiction detection between incoming and existing feed values at matching dimension+key. Conflicts recorded with PENDING status preserving both values. Integrated FROverallUpdateLog for complete mutation audit trail. Added asyncio.Semaphore(1) concurrency guard. All WP2.3 criteria met.
- ▶ [WP4.1 · 5h · 80%] **Lark WebSocket Connection & Identity Mapping** Established WebSocket connection layer for Lark Bot real-time communication with open_id-based user-to-FigureAndRelation mapping. Built event reception pipeline: text message receipt → user identity validation → FR resolution → conversation dispatch. Error recovery with automatic reconnection in progress. Foundation for all downstream WP4 features.

IV. Challenges & Solutions

- Issue:** LLM output inconsistency across dimensions (resolved)
 - PROCEDURAL_INFO extraction occasionally returned free-form text failing schema validation, while PERSONALITY/INTERACTION_STYLE outputs were consistently structured. → Added dimension-specific output format constraints and a two-pass extraction (structured generation → validation + gap-fill). Improved PROCEDURAL_INFO reliability from ~70% to >95%.

V. Key Deliverables

- ✓ FRBuildingGraph preprocessing node with 3 OriginalSourceType classifiers
- ✓ FineGrainedFeed extraction pipeline across 4 dimensions (16 distinct field types)
- ✓ FineGrainedFeedConflict table + detection logic with PENDING resolution workflow
- ✓ FROverallUpdateLog audit trail recording all FR mutations with source attribution
- ✓ Lark WebSocket connection layer with open_id user mapping and event routing
- ✓ Concurrency guard (Semaphore) preventing duplicate FRBuildingGraph execution

VI. Next Steps

- Complete WP4.1 WebSocket error recovery with exponential backoff reconnection (Week 8)
- Prepare and execute WP4.5 channel integration test cycle for all Lark API interactions
- Support WP2.5 review follow-ups: fine-tune extraction prompts based on accuracy data
- Coordinate recall dimension weights with Haojie Hu for ConversationGraph tuning
- Begin WP5.4 data layer regression test design for system testing phase (Week 9)

Yihan Liu

RBS Area: Conversation & Dialogue System (RBS R2.1) | Total Hours: 42h

I. WBS Task Breakdown

WBS Task	Description	Hours	Compl.	Status
WP3.1	ConversationGraph State & Parallel Feed Recall	14	100%	Complete
WP3.2	Memory Trimming & Rolling Summary	16	90%	In Progress
WP3.3	LLM Call & Dynamic Prompt Assembly	8	70%	In Progress
WP3.4	Feed Sync Node (Conversation-to-Persona Loop)	4	40%	In Progress

II. Period Summary

My work focused exclusively on WP3, the core intelligence engine of HeartCompass. WP3.1 delivered a fully functional ConversationGraph with typed StateGraph, parallel recall branches, and persona loading (14h, 100%). WP3.2 implemented round-based memory trimming with rolling LLM summaries and PostgresSaver checkpoint persistence, resolving a critical ORM serialization blocker (16h, 90%). WP3.3 initiated the LLM call node with dynamic prompt assembly from persona + recall + summary + messages (8h, 70%). WP3.4 began the feed sync node closing the conversation-to-persona loop (4h, 40%). Total: 42h across 4 WBS sub-tasks. The ConversationGraph pipeline is operational end-to-end with checkpointed state persistence across service restarts.

III. Key Progress by WBS Task

- ▶ [WP3.1 · 14h · 100%] **ConversationGraph State & Parallel Branch Architecture** Designed ConversationGraph using LangGraph StateGraph with typed input/output schemas. Built nodeLoadFRAndPersona assembling structured persona markdown from 4 core fields (excluding fields redundant with recalled FineGrainedFeed). Implemented parallel branch structure: semantic feed recall (MEMORY + PROCEDURAL_INFO via pgvector) alongside message history assembly — reducing end-to-end latency ~35% vs. sequential execution. Delivered 14h of core architecture, all milestones met.
- ▶ [WP3.2 · 16h · 90%] **Round-Based Memory Trimming & Checkpoint Persistence** Implemented memory management preventing context window overflow in extended dialogues. Enforces configurable thresholds (max 8000 chars / 40 messages) with round-completeness guarantee — partial HumanMessage+AIMessage pairs are never split. Trimmed content compressed into rolling conversation_summary via LLM and injected as system context. Resolved critical PostgresSaver ORM serialization failure with custom JSON encoder. State survives restarts. Remaining: long-session (>50 rounds) edge tuning.
- ▶ [WP3.3 · 8h · 70%] **LLM Call & Dynamic Prompt Assembly** Built nodeCallLLM assembling system prompt from 4 dynamic sources: persona markdown, recalled FineGrainedFeed, rolling conversation_summary, and current user messages. Designed 7 role-specific prompt templates (self/family/friend/mentor/colleague/partner/public-figure) adjusting tone and response patterns. Structured output parsing generates validated messages_to_send. Remaining: retry strategy with exponential backoff and end-to-end integration testing. Currently at 70%.
- ▶ [WP3.4 · 4h · 40%] **Feed Sync — Conversation-to-Persona Loop** Began the feed synchronization node closing the feedback loop between conversation outputs and persona data. New insights evaluated for confidence threshold: high-confidence, consistent observations update FR core fields; lower-confidence items become standalone FineGrainedFeed entries. Update decision logic integrates with WP2.3 FineGrainedFeedConflict system. Data flow design complete; full implementation targeted for Week 8. Currently at 40%.

IV. Challenges & Solutions

- Issue: PostgresSaver ORM serialization failure (resolved, May 9)
 - LangGraph PostgresSaver attempted direct JSON serialization of ORM entities, causing TypeError on datetime fields and relationship proxies — blocked WP3.2 for 3 days.
 - Implemented custom JSON encoder with entity-to-dict conversion, ISO datetime formatting, relationship traversal depth limits, and circular reference detection. Checkpoint save/restore now works for all conversation state components.

V. Key Deliverables

- ✓ ConversationGraph pipeline with typed StateGraph (5 nodes: LoadFR, RecallFeeds, BuildMessages, CallLLM, SyncFeeds)
- ✓ Parallel branch architecture for feed recall + message assembly (~35% latency reduction)
- ✓ Round-based memory trimming with LLM-powered rolling conversation_summary
- ✓ PostgresSaver checkpoint persistence with custom ORM entity JSON encoder
- ✓ Dynamic prompt assembly with 7 role-specific recipe templates
- ✓ Structured output parsing for LLM response generation with type validation

VI. Next Steps

- WP3.3: Complete LLM retry strategy (exponential backoff, max 3 attempts) and output validation (Week 8)
- WP3.4: Implement feed sync update decision logic with confidence scoring (Week 8)
- Prepare WP3.5 test scenarios: 20-turn coherence, persona consistency across sessions
- Optimize prompt templates for natural tone in extended (>30 turn) dialogues
- Investigate embedding-based message deduplication for redundant context prevention

Haojie Hu

RBS Area: Knowledge Base & Vector Recall (RBS R1.2.2, R2.1.1) | Total Hours: 40h

I. WBS Task Breakdown

WBS Task	Description	Hours	Compl.	Status
WP2.4	Vector Embedding Pipeline & Semantic Recall	20	100%	Complete
WP2.5	Persona Building Review & Quality Testing	12	100%	Complete
WP3.1	Recall Optimization for ConversationGraph	8	100%	Complete

II. Period Summary

My work in Weeks 5–7 completed the WP2 knowledge infrastructure and supported WP3.1 recall integration. WP2.4 (20h, 100%) delivered pgvector-based embedding generation and semantic recall with dimension-filtered queries achieving sub-500ms latency under HNSW index (m=16, ef_construction=200). WP2.5 (12h, 100%) conducted the systematic persona building review with test cases across all 5 FigureRole types and 3 OriginalSourceType inputs, confirming per-dimension accuracy: PERSONALITY 94%, INTERACTION_STYLE 91%, PROCEDURAL_INFO 88%, MEMORY 93%. WP3.1 recall optimization (8h, 100%) integrated vector recall as a parallel ConversationGraph branch with tuned dimension prioritization minimizing token consumption. Total: 40h across 3 WBS tasks.

III. Key Progress by WBS Task

- ▶ [WP2.4 · 20h · 100%] **Vector Embedding Generation & Semantic Recall Engine** Completed pgvector-based embedding storage with dimension-aware chunking (larger chunks for MEMORY narrative, smaller for PERSONALITY traits). Built semantic recall endpoint with dimension-filtered queries (restrict search to specific dimensions) returning top-K by cosine distance (K=5 default). Configured HNSW index (m=16, ef_construction=200) achieving <500ms query latency for 1K–10K entries. Added L2 normalization validation ensuring embedding consistency across service restarts.
- ▶ [WP2.5 · 12h · 100%] **Systematic Persona Building Review & Quality Validation** Designed comprehensive test cases: all 5 FigureRole types with role-specific extraction expectations; 3 OriginalSourceType inputs for source-aware preprocessing validation; edge cases (empty input, contradictory input, Chinese+English, >5000 words). Documented per-dimension accuracy: PERSONALITY 94%, INTERACTION_STYLE 91%, PROCEDURAL_INFO 88%, MEMORY 93%. Verified conflict detection with 0% false positives. Resolved list-type FR field partial update extraction inaccuracy via prompt adjustment.
- ▶ [WP3.1 · 8h · 100%] **Recall Optimization for ConversationGraph** Integrated vector recall as a parallel ConversationGraph branch with dimension prioritization: MEMORY + PROCEDURAL_INFO feeds for conversation grounding, while PERSONALITY + INTERACTION_STYLE draw from persona markdown — avoiding redundant LLM context injection. Verified <500ms recall contribution to end-to-end response time, meeting real-time conversation requirement. Collaborated with Yihan Liu on graph architecture and token budget optimization.

IV. Challenges & Solutions

- Issue:** HNSW index build time for large corpora (resolved)
 - Building HNSW index on 10K+ entries took >30s, causing unacceptable startup latency. → Implemented background index building with ready flag — service accepts queries via sequential scan immediately, switches to HNSW once background build completes. Startup reduced to <3s without sacrificing query performance.

V. Key Deliverables

- ✓ pgvector embedding storage with HNSW index (m=16, ef_construction=200, <500ms latency)
- ✓ Dimension-filtered semantic recall endpoint (top-K cosine distance, per-dimension restriction)
- ✓ Batch embedding generation with dimension-aware chunking strategy
- ✓ WP2.5 systematic review report with per-dimension accuracy metrics and test case library
- ✓ Recall-to-ConversationGraph integration with dimension prioritization for token efficiency
- ✓ Preliminary Docker Compose evaluation for WP5.3 preparation

VI. Next Steps

- Prepare vector recall benchmarks and test datasets for WP5.4 system testing (Weeks 9–10)
- Optimize HNSW parameters (m, ef_search) for larger-scale (50K+) feed corpora
- Support WP3.5 review: provide recall quality metrics and relevance assessments
- Begin Docker Compose configuration and Alembic migration scaffolding for WP5.3 (Week 9)
- Investigate hybrid search (semantic + keyword) for improved rare-term recall precision

Tian Bu (Team Leader)

RBS Area: CLI, Bot Commands & Project Governance (RBS R2.2.2, R3.1.1) | Total Hours: 38h

I. WBS Task Breakdown

WBS Task	Description	Hours	Compl.	Status
WP4.2	Bot Menu System & Persona Build Commands	14	85%	In Progress
WP4.3	Message Batching & Card Template Design	8	60%	In Progress
WP5.1	CLI Framework & Doctor/Setup Commands	12	75%	In Progress
—	Project Governance & WBS Milestone Tracking	4	—	Ongoing

II. Period Summary

My work spanned three development areas plus project governance. WP4.2 (14h, 85%) implemented the Lark Bot command menu — /menu, /list_available_persons, /show_persona, /build_persona — with structured card output and input validation. An initial card template field mapping issue was identified, reverted, and re-implemented. WP4.3 (8h, 60%) designed the message batching strategy with 15s configurable wait timer and per-user buffer isolation. WP5.1 (12h, 75%) scaffolded the immortality CLI with argparse command tree: doctor (environment validation), setup (env generation + DB init), log inspection, and --json output flag. Governance (4h): maintained WBS tracking, coordinated WP2.5 review cycle, updated Project Charter, compiled this report. Total: 38h.

III. Key Progress by WBS Task

- ▶ [WP4.2 · 14h · 85%] **Bot Menu System & Persona Interaction Commands** Designed and implemented the Lark Bot command menu: /menu displays available commands; /list_available_persons queries user's FR records with IDs, names, and relationship types; /show_persona displays structured persona summaries organized by 4 core dimensions; /build_persona accepts natural language input and triggers FRBuildingGraph pipeline with progress feedback. All commands include input validation and user-friendly error messages. Initial card template field mapping issues were reverted and re-implemented with corrected explicit field contracts. Edge-case handling for empty persona displays in progress.
- ▶ [WP4.3 · 8h · 60%] **Message Batching Strategy & Card Template System** Designed buffer-and-dispatch mechanism: incoming messages accumulated per-user in memory (keyed by open_id) with configurable 15s wait timer. Each new message resets the timer; on expiry, all batched messages dispatch as a single conversational turn. This reflects natural conversation rhythm where users send multiple short messages forming one thought. Began card template system: persona report cards with dimension-organized sections, build result notifications with progress indicators, conversation reply formatting. Continues through Week 8. Currently at 60%.
- ▶ [WP5.1 · 12h · 75%] **CLI Framework & System Management Commands** Scaffolded immortality CLI with argparse hierarchical subcommand routing. Doctor command: Python version check (3.11–3.13), env var verification (DATABASE_URL, LARK_APP_ID, LARK_APP_SECRET, DOUBAO_API_KEY), dependency integrity via import checks, DB connectivity test. Setup command: .env generation from template with interactive prompts, database table initialization via SQLAlchemy, pgvector extension verification. Log inspection subcommand with level/module filtering. --json flag for programmatic output consumption. Provides auditable system administration replacing direct DB access. Currently at 75%.
- ▶ [Governance · 4h · Ongoing] **Project Governance & WBS Milestone Tracking** Maintained WBS progress tracking dashboard with task-level completion mapped to weekly milestone table. Verified all scheduled WP2.1–WP2.5 and WP3.1 tasks completed and traceable to RBS areas. Coordinated WP2.5 review cycle: collected test results, verified issue resolution. Updated Project Charter to reflect current architecture and entity model. Compiled this bi-weekly report with WBS-anchored progress consolidation. Prepared Weeks 8–11 Git commit plan (95 commits across 4 members).

IV. Challenges & Solutions

- Issue:** Card template field mapping misalignment (resolved)
 - Initial /show_persona and /build_persona templates referenced FRBuildingGraphReport field names mismatched with actual output schema, causing blank card sections. → Reverted, audited all output fields against template references, re-applied with explicit field contracts in code. ~2 days rework but eliminated entire template-display bug class.

V. Key Deliverables

- ✓ Lark Bot command menu: /menu, /list_available_persons, /show_persona, /build_persona
- ✓ Message batching system with configurable 15s wait timer and per-user buffer isolation
- ✓ Card template system with explicit field contracts for persona reports and notifications
- ✓ immortality CLI: doctor, setup, log inspection subcommands with --json flag
- ✓ Updated Project Charter reflecting current codebase architecture and entity model
- ✓ Weeks 8–11 Git commit plan (95 commits across 4 members, 4 weeks of development)
- ✓ Third bi-weekly report with WBS-anchored progress tracking and completion metrics

VI. Next Steps

- WP4.3: Complete message batching edge cases — timer expiry during send, buffer overflow, cross-device dedup (Week 8)
- WP4.2: Add /delete_persona and /update_persona commands (Week 8)
- WP5.1: Complete setup with full .env template coverage; begin WP5.2 auth login (Week 8)
- Coordinate WP3.5 + WP4.5 review cycles — prepare criteria and test scenarios
- Maintain WBS milestone tracking and compile Week 8–9 checkpoint report